CS4100 Progress Report

Yuqi Chen

Project Background

I am building an Approximate Q-learning agent for playing the popular card game UNO for my final project. UNO provides an interesting yet challenging reinforcement learning environment to experiment with as It includes partial observability, stochasticity and multi-agent competition. I plan to add to existing UNO environments by implementing popular custom cards, adding 4 new cards to the game and extending strategy. Finally, I plan to evaluate my final Q-learning agent against a simple random agent, a rule based agent, a heuristic agent, and finally myself, a human player.

Progress Update

Since my project proposal, I have changed the underlying project environment from Pfann's UNO github repo[1] to RLCard[2] . RLCard's UNO environment is cleanly packaged, therefore suitable for my extension of 4 common custom cards. While inspecting RLCard's environment, I found several rule inconsistencies and one major drawback. The inconsistencies to UNO's official rules include automatically playing a drawn card of the same color, not letting players decide; Only allow Draw 4 when no other playable cards exist. Uno's official rules state that Draw 4 can be played when there are no playable cards of the same color, and their original rules ruled out legal moves. Fixes for these inconsistencies were implemented. Surprisingly, a major drawback exists that the environment doesn't allow a voluntary draw in a turn, an important strategic element of UNO. It forces the player to play a card each turn. Voluntary drawing is allowed by the official rule, and voluntarily drawing a card to preserve powerful cards, such as a draw 4, could be a gaming winning move. I changed his original implementation to allow voluntary drawing. Finally, I implemented the logic of my 4 custom cards, shuffle redistribute, draw until, discard color, and swap deck.

On the learning side, I have built a random agent that acts as a baseline. It will pick from available actions completely randomly. It serves as a baseline that all future agents should beat. As such, comparisons can't be made yet that show signs of learning.

Since I have been focused on building the environment, for my next steps, I plan to implement the rule based agent, the heuristics agent, and learn more about approximate Q-learning. The state space of card games is too large for traditional Q-learning which necessitates functional approximation of Q values. Implementation of the Q-learning agent will start after.

References

[1] B. Pfann, "uno-card-game-rl," GitHub repository, commit 5608371, [year]. [Online]. Available: https://github.com/bernhard-pfann/uno-card-game-rl. Accessed: Oct. 30, 2026.

[2] D. Zha et al., "RLCard: A toolkit for reinforcement learning in card games," Version 1.2.0, [Software]. [Online]. Available: https://github.com/datamllab/rlcard. Accessed: Oct. 30, 2026.